

UNIT 1: INTRODUCTION OF PROBLEM SOLVING

Complete Study Notes

1. Computational Thinking

Computational Thinking (CT) is a fundamental skill for everyone, not just for computer scientists. It involves solving problems, designing systems, and understanding human behavior, by drawing on the concepts fundamental to computer science. It is the thought process involved in formulating a problem and expressing its solution(s) in such a way that a computer (human or machine) can effectively carry it out.

1.1 Objectives of Computational Thinking

The primary objectives of adopting computational thinking include:

- **Problem Formulation:** To enable individuals to articulate problems clearly in a way that allows for the use of computers and other tools to help solve them.
- **Logical Organization:** To logically organize and analyze data to find the most efficient path to a solution.
- **Abstraction and Representation:** To represent data through abstractions such as models and simulations.
- **Automation:** To automate solutions through algorithmic thinking (a series of ordered steps).
- **Efficiency:** To identify, analyze, and implement possible solutions with the goal of achieving the most efficient and effective combination of steps and resources.

1.2 Outcomes of Computational Thinking

When an individual successfully applies computational thinking, the expected outcomes are:

- **Decomposition Skill:** The ability to break down a complex problem into smaller, more manageable parts.
- **Pattern Recognition:** The ability to notice similarities, differences, trends, and patterns in data.
- **Algorithmic Design:** The capacity to develop step-by-step instructions for solving a problem.
- **Generalization (Abstraction):** The ability to identify general principles that generate these patterns and strip away unnecessary details.
- **Confidence in Complexity:** Increased confidence in dealing with ambiguous or complex, open-ended problems.
- **Adaptability:** The ability to transfer problem-solving processes to a wide variety of

different problems across different disciplines.

1.3 How is Computational Thinking Used?

Computational thinking is used by applying its four core pillars to any given problem:

1. **Decomposition:** Breaking a large task down. *Example:* Writing a large software program is broken down into writing individual functions and modules.
2. **Pattern Recognition:** Looking for similarities. *Example:* If a doctor notices that patients with a certain disease all have similar symptoms, they use pattern recognition to diagnose future patients faster.
3. **Abstraction:** Focusing on the important information only, ignoring irrelevant detail. *Example:* A map of a city is an abstraction; it shows roads and landmarks but ignores the exact number of trees or the color of houses.
4. **Algorithms:** Developing the step-by-step instructions to solve the problem. *Example:* Creating a recipe to bake a cake, or writing the actual code to sort a list of numbers.

2. Concepts of Logical Thinking

Logical thinking is the act of analyzing a situation and coming up with a sensible solution. In computer science, computers do not have common sense; they rely entirely on the precise logic provided by the programmer.

Core Concepts:

- **Deductive Reasoning:** Starting with a general rule and applying it to a specific case to reach a certain conclusion (e.g., All computers use binary -> A laptop is a computer -> Therefore, a laptop uses binary).
- **Inductive Reasoning:** Observing specific cases to form a general rule or hypothesis (often used in data science and machine learning).
- **Boolean Logic:** The foundation of modern computer logic, dealing with truth values: True (1) or False (0). It uses operators like AND, OR, and NOT to make decisions.
- **Conditional Logic (If-Then-Else):** Making decisions based on certain conditions. If a condition is true, execute Action A; otherwise, execute Action B.
- **Iterative Logic (Loops):** Repeating a set of actions until a specific logical condition is met.

3. Algorithms

An algorithm is a finite, well-defined sequence of step-by-step instructions designed to solve a specific problem or perform a specific task. Algorithms are independent of any programming language.

Characteristics of a Good Algorithm:

1. **Input:** An algorithm should have zero or more well-defined inputs.
2. **Output:** An algorithm should produce one or more well-defined outputs, matching the

desired result.

3. **Definiteness:** Each step must be clear, unambiguous, and precisely defined.
4. **Finiteness:** The algorithm must terminate after a finite number of steps. It cannot run infinitely.
5. **Effectiveness:** Every step must be basic enough that it can, in principle, be carried out exactly and in a finite amount of time by a person using a pencil and paper.

Example Algorithm (To find the largest of two numbers, A and B):

- Step 1: Start
- Step 2: Read two numbers, A and B.
- Step 3: If $A > B$, then go to Step 4, else go to Step 5.
- Step 4: Print "A is the largest" and go to Step 6.
- Step 5: Print "B is the largest".
- Step 6: Stop.

4. Flow Charts

A flow chart is a graphical or pictorial representation of an algorithm. It uses different geometric shapes to denote different types of instructions and arrows to show the direction of the flow of control.

Standard Flow Chart Symbols:

- **Oval (Terminal):** Used to indicate the Start and Stop/End of the process.
- **Parallelogram (Input/Output):** Used to denote inputting data (e.g., Read X) or outputting data (e.g., Print Y).
- **Rectangle (Process/Action):** Used to represent mathematical operations, variable assignments, or general data processing (e.g., $\text{Sum} = A + B$).
- **Diamond (Decision):** Used to represent a conditional operation that dictates the path of the program based on a Yes/No or True/False answer (e.g., Is $X > Y$?).
- **Arrows (Flowlines):** Show the sequence of steps and the direction of logic flow.
- **Circle (Connector):** Used to connect different parts of a complex flow chart across pages.

Advantages of Flow Charts:

- **Visual Clarity:** Makes complex logic easier to understand.
- **Effective Communication:** Excellent tool for communicating program logic to non-technical stakeholders.
- **Debugging Aid:** Helps in finding logical errors before coding begins.
- **Documentation:** Serves as good documentation for future reference.

5. Objectives of Problem Solving

In computer science, problem-solving is the systematic approach to identifying a problem and

designing an optimal solution that a computer can execute.

The core objectives are:

- To find a solution that is **correct** (gives the right output for all valid inputs).
- To find a solution that is **efficient** (uses minimal time and memory).
- To create a solution that is **scalable** (can handle an increasing amount of data).
- To ensure the solution is **maintainable** (easy to read, modify, and update later).
- To build a solution that is **robust** (can handle unexpected inputs or errors gracefully).

6. Defining a Problem

Before a problem can be solved, it must be thoroughly understood and defined. A poorly defined problem leads to the wrong solution.

Steps to Define a Problem:

1. **Identify the Core Issue:** What exactly needs to be solved? Strip away the "noise."
2. **Determine Inputs:** What data or information is available or needs to be collected to start solving the problem?
3. **Determine Outputs:** What is the exact expected outcome, result, or format of the solution?
4. **Identify Constraints:** What are the limitations? (e.g., time limits, memory limits, specific hardware requirements, budget).
5. **Identify Edge Cases:** What are the extreme or unusual inputs that the system must handle?

7. Devising Solutions

Once the problem is defined, the next step is devising a solution (Algorithm Design).

Steps in Devising Solutions:

1. **Brainstorming:** Generating multiple potential approaches to the problem.
2. **Evaluating Alternatives:** Analyzing the pros and cons of each approach based on efficiency, ease of implementation, and constraints.
3. **Top-Down Design (Stepwise Refinement):** Starting with the big picture and breaking it down into smaller, solvable sub-problems.
4. **Writing the Algorithm / Flowchart:** Documenting the chosen solution logically.
5. **Desk Checking (Dry Run):** Manually tracing the algorithm with sample data to verify its correctness before writing any code.

8. Conceptual Introduction: Topics in Computer Science Algorithms

As you dive deeper into computer science, you encounter standard categories of algorithms designed to solve common computational problems.

Key Algorithm Topics:

- **Searching Algorithms:** Designed to retrieve information stored within some data structure. Examples: *Linear Search* (checking items one by one), *Binary Search* (efficiently finding an item in a sorted list by repeatedly dividing the search interval in half).
- **Sorting Algorithms:** Designed to arrange elements in a list in a certain order (ascending or descending). Examples: *Bubble Sort*, *Selection Sort*, *Insertion Sort*, *Merge Sort*, *Quick Sort*.
- **Optimization Algorithms:** Used to find the "best" solution from all feasible solutions (e.g., finding the shortest path between two cities, like *Dijkstra's Algorithm*).
- **Algorithm Complexity (Time & Space):** Studying how the runtime (time complexity) and memory usage (space complexity) of an algorithm grow as the input size grows. This is often denoted using "Big O notation" (e.g., $O(n)$, $O(n^2)$).

9. Modern Computer Systems: Hardware Architecture

Modern computers are largely based on the **Von Neumann Architecture**, proposed by John von Neumann in 1945. It defines a computer system with a central processing unit, memory, and input/output mechanisms.

Key Hardware Components:

1. **Central Processing Unit (CPU):** The "brain" of the computer. It consists of:
 - **Arithmetic Logic Unit (ALU):** Performs all mathematical calculations (addition, subtraction) and logical operations (AND, OR, comparisons).
 - **Control Unit (CU):** Fetches instructions from memory, decodes them, and directs the ALU and other components to execute them. It manages the flow of data.
 - **Registers:** Extremely fast, small memory locations inside the CPU used to store temporary data and instructions currently being processed.
2. **Memory (Primary Storage):**
 - **RAM (Random Access Memory):** Volatile memory used to store data and programs currently in use by the CPU. Loses data when power is turned off.
 - **ROM (Read-Only Memory):** Non-volatile memory that stores critical system instructions (like the BIOS) needed to boot the computer.
 - **Cache Memory:** High-speed memory located close to the CPU to store frequently accessed data, speeding up processing.
3. **Secondary Storage:** Non-volatile memory for long-term storage (e.g., Hard Disk Drives (HDD), Solid State Drives (SSD), Flash Drives).
4. **Input/Output (I/O) Devices:**
 - * *Input:* Keyboard, mouse, scanner, microphone (feed data into the system).
 - *Output:* Monitor, printer, speaker (present processed data to the user).

5. **System Bus:** The communication channels (wires/traces) that transfer data, memory addresses, and control signals between the CPU, memory, and I/O devices.

10. Data Representation in Computers

Computers do not understand human languages, letters, or decimal numbers directly. They operate entirely using electricity, which can be in two states: ON or OFF. Therefore, all data must be represented in binary.

Key Concepts in Data Representation:

- **Bits and Bytes:** * A **Bit** (Binary Digit) is the smallest unit of data, represented as a 0 (off) or 1 (on).
 - A **Byte** is a sequence of 8 bits. It is the basic unit used to represent a single character of text.
- **Number Systems:**
 - **Binary System (Base 2):** Uses only 0 and 1. (Used by hardware).
 - **Decimal System (Base 10):** Uses 0-9. (Used by humans).
 - **Octal (Base 8) & Hexadecimal (Base 16):** Used in computer science as a shorthand for binary. Hexadecimal uses 0-9 and A-F.
- **Representing Text (Character Encoding):**
 - **ASCII (American Standard Code for Information Interchange):** Uses 7 or 8 bits to represent English letters, numbers, and symbols. For example, 'A' is represented as 65 (binary: 01000001).
 - **Unicode:** A modern standard that uses up to 32 bits, capable of representing text in almost all the world's writing systems, including emojis.
- **Representing Numbers:**
 - *Integers* are represented using direct binary translation. Negative numbers are often represented using a method called **Two's Complement**.
 - *Real Numbers (Fractions/Decimals)* are represented using **Floating-Point representation** (IEEE 754 standard), which stores a number as a sign, an exponent, and a mantissa (similar to scientific notation).
- **Representing Images and Sound:** Images are divided into tiny dots called pixels, with each pixel's color represented by binary numbers. Sound is captured by sampling the audio wave at intervals and representing the amplitude as binary numbers.

11. Software and Operating System

Hardware is useless without instructions to tell it what to do. Software provides these instructions.

11.1 Software

Software is a collection of data, programs, procedures, and routines that tell a computer how

to work.

- **System Software:** Programs that manage the hardware and provide a platform for other software to run. (e.g., Operating Systems, Device Drivers, Utility Programs).
- **Application Software:** Programs designed to perform specific tasks for the user. (e.g., Web browsers, Microsoft Word, Games, Video editors).

11.2 Operating System (OS)

An Operating System is the most critical piece of system software. It acts as an intermediary or interface between the user, the application software, and the computer hardware. Examples: Microsoft Windows, macOS, Linux, Android, iOS.

Primary Functions of an Operating System:

1. **Process Management:** The OS decides which program gets to use the CPU, when, and for how long. It manages the execution of multiple applications simultaneously (multitasking).
2. **Memory Management:** The OS keeps track of primary memory (RAM). It allocates memory space to programs when they start and frees it up when they close, ensuring one program doesn't accidentally overwrite another's data.
3. **File Management:** The OS organizes data into files and directories (folders) on storage devices. It handles reading, writing, moving, and deleting files, as well as enforcing file access permissions.
4. **Device Management (I/O Management):** The OS uses device drivers to communicate with hardware peripherals (keyboards, printers, network cards). It handles the input and output processes smoothly.
5. **User Interface (UI):** The OS provides a way for users to interact with the computer. This can be a Command-Line Interface (CLI) where commands are typed, or a Graphical User Interface (GUI) with windows, icons, and menus.
6. **Security and Protection:** The OS uses passwords and user accounts to prevent unauthorized access to the system and protects data from malicious software.

UNIT 2: BASICS OF PYTHON

Complete Study Notes

1. Python Interface

Python can be run in different ways depending on what you are trying to accomplish. The interface refers to how a programmer interacts with the Python interpreter.

- **Interactive Mode (REPL):** REPL stands for Read, Evaluate, Print, Loop. You can type Python code directly into the command prompt or terminal, and the interpreter instantly executes it and shows the result. It is great for testing quick snippets of code.
- **Script Mode:** For larger programs, you write Python code in a file with a .py extension (a script) using a text editor or an Integrated Development Environment (IDE like PyCharm, VS Code, or IDLE). You then run the entire file at once.

2. When to use Python

Python is a high-level, interpreted, general-purpose language known for its readability. It should be used for:

- **Web Development:** Backend development using frameworks like Django or Flask.
- **Data Science & Machine Learning:** It is the industry standard for data analysis, AI, and ML, using libraries like Pandas, NumPy, and TensorFlow.
- **Automation and Scripting:** Automating repetitive, boring tasks (e.g., file moving, web scraping).
- **Software Prototyping:** Its simple syntax allows developers to build functional prototypes much faster than in C++ or Java.

3. Comments

Comments are text in a program that the Python interpreter completely ignores. They are used to explain the code and make it more readable for humans.

- **Single-line Comments:** Start with a hash symbol (#).

Python

```
# This is a single-line comment  
print("Hello") # This prints a greeting
```

- **Multi-line Comments (Docstrings):** Python doesn't have an explicit multi-line comment syntax like /* ... */, but unassigned string literals are used for this purpose, enclosed in triple quotes (""" or ''').

Python

```
"""  
This is a multi-line comment.  
It spans multiple lines.  
"""
```

4. Core Basics

4.1 Literal Constants

A literal constant is a value that is explicitly typed into the code and never changes its underlying value.

- Examples: 5, 1.23, 'Hello', True. The number 5 will always be 5.

4.2 Numbers

Python supports three main numeric types:

- **Integer (int):** Whole numbers, positive or negative, without decimals. Example: `x = 10`
- **Floating point (float):** Numbers with a decimal point or in exponential form. Example: `y = 3.14` or `z = 2.5e2`
- **Complex numbers (complex):** Written with a `"j"` as the imaginary part. Example: `c = 3 + 4j`

4.3 Variables

A variable is a named container used to store data in the computer's memory. Unlike languages like C or Java, Python variables do not need explicit declaration to reserve memory space. The declaration happens automatically when you assign a value to a variable using the `=` sign.

Python

```
age = 20          # 'age' is a variable holding an integer  
name = "Alice"    # 'name' is a variable holding a string
```

4.4 Identifier Naming

Identifiers are names given to variables, functions, classes, etc. Python has strict rules for naming them:

1. Must start with a letter (A-Z or a-z) or an underscore (`_`).
2. Cannot start with a number.
3. Can contain letters, numbers, and underscores (A-z, 0-9, and `_`).
4. **Case-sensitive:** `myVar` and `myvar` are two different identifiers.
5. **Reserved Words:** You cannot use Python keywords (like `if`, `for`, `while`, `class`) as identifier names.

4.5 Data Types

Every value in Python has a datatype. Since everything is an object in Python, data types are actually classes.

- **Numeric:** int, float, complex
- **Sequence Type:** str (string), list, tuple
- **Boolean:** bool (True or False)
- **Set:** set (unordered collection of unique items)
- **Dictionary:** dict (key-value pairs)
- *You can check a variable's type using the `type()` function.*

4.6 Strings

A string is a sequence of characters enclosed in single (' '), double (" "), or triple (''' ' or "" "" ""'") quotes. Strings in Python are **immutable**, meaning once a string is created, the characters within it cannot be changed.

```
Python
greeting = "Hello World"
```

5. Input / Output

Basic I/O Statements

- **Output (`print()`):** Used to display information to the console.

```
Python
print("Hello, Python!")
name = "Bob"
print("My name is", name) # Output: My name is Bob
```

- **Input (`input()`):** Used to take input from the user. By default, `input()` **always returns a string**.

```
Python
user_name = input("Enter your name: ")
```

- **Typcasting Input:** If you need a number from the user, you must convert (cast) the string input into an int or float.

```
Python
age = int(input("Enter your age: ")) # Converts the string input to
an integer
```

6. Program Structure (CRITICAL CONCEPTS)

6.1 Logical and Physical Line (Often Missed!)

It is extremely important to understand how Python reads lines of code.

- **Physical Line:** This is what you actually see when you write the code. It is the text ending with the "Enter" or "Return" key.
- **Logical Line:** This is a single, complete statement as the Python interpreter understands it.
- *Python assumes that each physical line corresponds to a single logical line.*

Joining Lines (Explicit and Implicit):

What if a logical line is too long to fit on one physical line?

1. **Explicit Line Joining:** You can use a backslash (\) at the end of a physical line to tell Python that the logical line continues on the next physical line.

```
Python
total_sum = 10 + 20 + \
            30 + 40
```

2. **Implicit Line Joining:** Expressions enclosed in parentheses (), brackets [], or curly braces {} can span multiple physical lines without using a backslash. Python automatically knows the logical line hasn't ended until the bracket is closed.

```
Python
my_list = [
    1, 2, 3,
    4, 5, 6
]
```

6.2 Indentation

Unlike C++ or Java which use curly braces {} to define blocks of code (like loops or functions), Python uses **whitespace indentation**.

- A block of code begins when the indentation increases and ends when the indentation decreases.
- The standard practice is to use **4 spaces** for each level of indentation.
- Mixing tabs and spaces can cause IndentationError.

7. Operations

Operators and Expressions

An **expression** is a combination of values, variables, and operators that evaluate to a single value (e.g., 5 + 3).

- **Arithmetic Operators:** + (addition), - (subtraction), * (multiplication), / (division - returns float), // (floor division - returns integer), % (modulus - returns remainder), ** (exponentiation).
- **Comparison Operators:** Used to compare values and return a Boolean (True or False). == (equal), != (not equal), > (greater than), < (less than), >=, <=.
- **Logical Operators:** Used to combine conditional statements: and, or, not.
- **Assignment Operators:** Used to assign values to variables. =, +=, -=, *=, (e.g., x += 5 is the same as x = x + 5).
- **Membership Operators:** in, not in (checks if a value exists in a sequence like a list or string).

8. Data Structures (IMPORTANT)

Data structures are ways to organize and store data so they can be accessed and worked with efficiently.

8.1 Lists

A list is a collection which is **ordered and mutable (changeable)**. Allows duplicate members. Created using square brackets [].

- **Indexing:** Accessing elements by their position. Python uses zero-based indexing (the first item is at index 0). Python also supports negative indexing (the last item is at index -1).

Python

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0]) # Output: apple
print(fruits[-1]) # Output: cherry
```

- **Slicing:** Extracting a portion of the list using the syntax [start : stop : step]. Note that the stop index is *exclusive*.

Python

```
nums = [0, 1, 2, 3, 4, 5]
print(nums[1:4]) # Output: [1, 2, 3]
print(nums[::-1]) # Output: [5, 4, 3, 2, 1, 0] (Reverses the list)
```

- **Basic Methods:**
 - .append(item): Adds an item to the end.
 - .insert(index, item): Inserts an item at a specific index.
 - .remove(item): Removes the first occurrence of the item.
 - .pop(index): Removes and returns the item at the given index (or the last item if no index is given).

8.2 Tuples (Often Missed!)

A tuple is a collection which is **ordered and IMMUTABLE (unchangeable)**. Allows duplicate members. Created using parentheses ().

- **Why use Tuples instead of Lists?** Because they cannot be changed, tuples are faster than lists and protect data from accidental modification. They can also be used as keys in dictionaries, whereas lists cannot.
- *Example:*

```
Python
coordinates = (10, 20)
print(coordinates[0]) # Output: 10
# coordinates[0] = 15 # ERROR! Tuples do not support item
assignment
```

8.3 Dictionary

A dictionary is a collection which is **ordered (as of Python 3.7), mutable, and does not allow duplicate keys**. It stores data in **key-value pairs**, allowing fast lookups. Created using curly braces {}.

- *Example:*

```
Python
student = {
    "name": "John",
    "age": 21,
    "course": "Computer Science"
}
print(student["name"]) # Output: John
```

- **Dictionary Methods:**
 - `.keys()`: Returns a list of all keys.
 - `.values()`: Returns a list of all values.
 - `.items()`: Returns a list of tuples containing the key-value pairs.

9. Control Flow

Control flow allows you to dictate the order in which the computer executes instructions based on conditions and repetitions.

9.1 Conditionals

Used to execute blocks of code only if a specific condition is met.

- **if statement:** Evaluates a condition. If True, executes the block.
- **elif (Else If) statement:** Used to check multiple conditions sequentially.
- **else statement:** Executes if all previous if/elif conditions evaluate to False.

Python

```
marks = 85
if marks >= 90:
    print("Grade: A")
elif marks >= 80:
    print("Grade: B")
else:
    print("Grade: C")
```

9.2 Iterators (Loops)

Used to repeat a block of code multiple times.

- **while Loop:** Repeats a block of code as long as a specified condition remains True.

Python

```
count = 0
while count < 3:
    print("Count is:", count)
    count += 1
```

- **for Loop:** Used to iterate over a sequence (like a list, tuple, dictionary, or string) or a range of numbers.

Python

```
# Iterating over a list
for fruit in ["apple", "banana"]:
    print(fruit)

# Iterating over a range (generates numbers 0 to 4)
for i in range(5):
    print(i)
```

- **Control Statements in loops:**
 - **break:** Exits the entire loop immediately.
 - **continue:** Skips the rest of the current iteration and moves to the next iteration of the loop.

UNIT 3: FUNCTIONS AND MODULES

Complete Study Notes

1. Introduction to Functions

A function is a reusable block of code designed to perform a specific task. Functions help break our program into smaller, modular chunks, making the code more organized, manageable, and avoiding repetition (DRY - Don't Repeat Yourself).

1.1 Function Parameters

Parameters are the variables listed inside the parentheses in the function definition. They act as placeholders for the data (arguments) that will be passed into the function when it is called.

Python

```
def greet(name):  
    print("Hello", name)  
greet("Alice")  
# Output: Hello Alice
```

1.2 Local Statement (Scope)

Variables defined *inside* a function body have a **local scope**. They can only be accessed and modified within that specific function. Once the function finishes executing, the local variables are destroyed.

Python

```
def my_func():  
    x = 10  
    print("Inside function:", x)  
my_func()  
# Output: Inside function: 10
```

1.3 Global Statement

Variables defined *outside* of all functions have a **global scope**. They can be read by any function. However, if you want to *modify* a global variable from inside a function, you must use the `global` keyword. Without it, Python will just create a new local variable with the same name.

Python

```
counter = 0
def increment():
    global counter
    counter += 1
increment()
print(counter)
# Output: 1
```

2. Arguments

When you call a function, the values you pass to it are called arguments. Python provides several flexible ways to pass arguments.

2.1 Default Argument Values

You can provide default values for parameters. If the caller does not pass an argument for that parameter, the default value is used.

Rule: Default arguments must always be placed at the end of the parameter list.

Python

```
def say_hello(name, message="Good morning!"):
    print(f"Hello {name}, {message}")
say_hello("John")
# Output: Hello John, Good morning!
say_hello("Jane", "How are you?")
# Output: Hello Jane, How are you?
```

2.2 Keyword Arguments

When calling a function, you can specify arguments by the names of their corresponding parameters. This allows you to skip arguments or place them out of order because the Python interpreter uses the keywords to match the values with parameters.

Python

```
def describe_pet(animal_type, pet_name):
    print(f"I have a {animal_type} named {pet_name}.")
describe_pet(pet_name="Harry", animal_type="hamster")
# Output: I have a hamster named Harry.
```

2.3 VarArgs Parameters (Variable-length Arguments)

Sometimes you don't know in advance how many arguments will be passed into a function. Python handles this using `*args` and `**kwargs`.

- `*args` (**Non-Keyword Arguments**): Collects extra positional arguments into a **tuple**.
- `**kwargs` (**Keyword Arguments**): Collects extra keyword arguments into a **dictionary**.

Python

```
def print_everything(*args, **kwargs):
    print("Positional args (Tuple):", args)
    print("Keyword args (Dictionary):", kwargs)
print_everything(1, 2, 3, a="apple", b="banana")
# Output: Positional args (Tuple): (1, 2, 3)
# Output: Keyword args (Dictionary): {'a': 'apple', 'b': 'banana'}
```

3. Output

Return Statement

The return statement is used to exit a function and pass a result back to the caller.

- A function can return any type of object (numbers, strings, lists, or even other functions).
- If a function does not have a return statement, or just says return without a value, it implicitly returns None.

Python

```
def add(a, b):
    result = a + b
    return result
sum_value = add(5, 3)
print(sum_value)
# Output: 8
```

4. Advanced Function Concepts

4.1 Recursive Functions

A recursive function is a function that calls itself during its execution. It is often used to solve problems that can be broken down into smaller, identical problems (like factorials or the Fibonacci sequence).

- **Base Case:** A condition that stops the recursion. Without it, the function would call itself infinitely, leading to a RecursionError.
- **Recursive Case:** The part where the function calls itself.

Python

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
print(factorial(5))  
# Output: 120
```

4.2 Docstrings (Documentation Strings)

A docstring is a string literal that occurs as the very first statement in a function, module, class, or method definition. It is used to document what the code does. It is usually written using triple quotes `""" """`.

You can access a function's docstring using the `__doc__` attribute or the `help()` function.

Python

```
def multiply(a, b):  
    """This function multiplies two numbers and returns the  
    result."""  
    return a * b  
print(multiply.__doc__)  
# Output: This function multiplies two numbers and returns the  
result.
```

5. Compilation (Often Missed!)

Byte Compiled (.pyc files)

Python is an interpreted language, but to speed up execution, the Python interpreter compiles the source code (.py files) into an intermediate, platform-independent format called **bytecode**.

- **What are they?** These bytecode files have a .pyc extension.
- **Where are they stored?** They are usually stored in a hidden directory named `__pycache__`.
- **Why does Python do this?** The compilation step is done to **speed up module loading times**. If a script is imported as a module, Python checks if a .pyc file exists and if the source code hasn't changed since the .pyc file was created. If so, it loads the .pyc file directly, bypassing the compilation step, making the program start faster.
- *Note:* .pyc files do not make the program *run* faster once it's loaded, they only make the initial *loading* process faster.

6. Modules

A module is simply a file containing Python code (variables, functions, classes) that you can include in your project. It helps organize large codebases.

6.1 Making Your Own Module

To create a module, just save your Python code in a file with a .py extension.

Example: Save this as math_utils.py

Python

```
def add(x, y):  
    return x + y
```

6.2 The from...import Statement

You can import an entire module using `import module_name`, but if you only need specific functions or variables, you use the `from...import` statement. This saves memory and prevents you from having to type the module name repeatedly.

Python

```
from math import pi, sqrt  
print(pi)  
# Output: 3.141592653589793  
print(sqrt(16))  
# Output: 4.0
```

6.3 Module's Name (__name__) (Often Missed!)

Every Python module has a built-in attribute called `__name__`. This is a very powerful concept used to determine how a script is being executed.

- If a script is being run **directly** (e.g., `python my_script.py`), Python sets the `__name__` variable to the string `"__main__"`.
- If a script is being **imported** into another script (e.g., `import my_script`), Python sets the `__name__` variable to the actual name of the module (the file name without .py).

Why is this useful?

It allows you to write code that only runs when the script is executed directly, and NOT when it is imported by someone else.

Python

```
def do_something():
    print("Doing something...")
if __name__ == "__main__":
    print("Testing the module directly.")
    # Output: Testing the module directly.
    do_something()
    # Output: Doing something...
```

7. Utility (Often Missed!)

The dir() Function

The `dir()` built-in function is an incredibly useful utility in Python. It returns a sorted list of strings containing the names of all the attributes (functions, variables, classes) defined within an object or a module.

- **When used on a module:** It shows everything that the module contains.
- **When used without arguments:** It shows the names of variables and functions currently defined in your local scope.

Python

```
import math
print(dir(math))
# Output: ['__doc__', '__name__', 'cos', 'pi', 'sin', 'sqrt', ...]
```

8. Packages

While modules help organize code into files, **Packages** help organize modules into directories.

- A package is basically a folder (directory) that contains multiple Python modules (.py files).
- **The `__init__.py` file:** Historically (and structurally), a directory must contain a special file named `__init__.py` for Python to recognize it as a package. This file can be empty, or it can contain initialization code for the package. (Note: Since Python 3.3, implicit namespace packages are supported, making `__init__.py` strictly optional, but it is still standard practice for regular packages).

Example Directory Structure:

```
my_game/ (Package)
  __init__.py
  levels.py (Module)
  characters.py (Module)
  weapons/ (Sub-package)
```

```
__init__.py  
guns.py
```

You can import from packages using dot notation:

```
import my_game.characters  
from my_game.weapons import guns
```

UNIT 4: CLASSES AND OBJECTS

Complete Study Notes

1. Basics of Object-Oriented Programming (OOP)

Object-Oriented Programming is a programming paradigm that revolves around the concept of "objects," which can contain data (attributes) and code (methods).

1.1 Defining Your Own Classes

A **class** is a blueprint or a template for creating objects. It defines a set of attributes and methods that will characterize any object that is instantiated from this class. Classes are created using the class keyword.

Python

```
# Defining a simple class
class Dog:
    pass # 'pass' is a placeholder when the class has no content yet
```

1.2 Creating Objects

An **object** is an instance of a class. Once a class is defined, you can create multiple independent objects from it. This process is called **instantiation**.

Python

```
class Car:
    pass

# Creating objects (instances) of the Car class
car1 = Car()
car2 = Car()
```

2. Constructors and Methods

2.1 The self Parameter

Inside a class, you will define functions, which are called **methods**. The first parameter of any standard instance method must always be self.

- self represents the specific object (instance) that is calling the method.

- It binds the attributes with the given arguments so that each object maintains its own independent state.

2.2 Constructors (__init__)

A constructor is a special method used to initialize a newly created object. In Python, the constructor is always named `__init__()`. It is called automatically the moment an object is created.

Python

```
class Student:
    # Constructor
    def __init__(self, name, roll_no):
        self.name = name        # Assigning data to the object's
        self.roll_no = roll_no  attribute

# When this line runs, __init__ is called automatically
student1 = Student("Alice", 101)
print(student1.name)
# Output: Alice
```

2.3 Explicit Constructor Invocation

Usually, Python calls `__init__` automatically. However, there are scenarios (especially in inheritance) where you need to call a constructor manually or explicitly.

- You can call a superclass's constructor explicitly using `SuperClassName.__init__(self)` or the preferred `super().__init__()`.

Python

```
class Parent:
    def __init__(self):
        print("Parent initialized")

class Child(Parent):
    def __init__(self):
        Parent.__init__(self) # Explicitly invoking the parent's
        print("Child initialized")  constructor

Child()
# Output: Parent initialized
# Output: Child initialized
```

2.4 Class Methods

While instance methods (using `self`) operate on a specific object, **class methods** operate on the class itself. They are shared across all instances.

- They are defined using the `@classmethod` decorator.
- Their first parameter is always `cls` (representing the class), not `self`.

Python

```
class Employee:
    company_name = "TechCorp" # Class variable

    @classmethod
    def change_company(cls, new_name):
        cls.company_name = new_name # Modifies the class variable
        directly

Employee.change_company("InnovateTech")
print(Employee.company_name)
# Output: InnovateTech
```

3. Attributes

Attributes are variables that belong to a class or an object.

3.1 Instance Variables

Instance variables are specific to a single object. They are defined inside methods (usually inside `__init__`) and are prefixed with `self`. If you change an instance variable in one object, it does not affect other objects.

Python

```
class Cat:
    def __init__(self, color):
        self.color = color # Instance variable

cat1 = Cat("Black")
cat2 = Cat("White")
print(cat1.color)
print(cat2.color)
# Output: Black
# Output: White
```

3.2 Class Variables

Class variables are shared by all objects of a class. They are defined directly inside the class body, outside of any methods.

Python

```
class Circle:
    pi = 3.14159 # Class variable

    def __init__(self, radius):
        self.radius = radius # Instance variable

print(Circle.pi) # Accessed using the class name
# Output: 3.14159
```

4. Attribute Handling

Python is a dynamic language, meaning you can manipulate an object's structure even after it has been created.

4.1 Adding Attributes

You can add new attributes to an existing object on the fly. You can do this using standard dot notation or the setattr() built-in function.

Python

```
class Book:
    pass

b1 = Book()
# 1. Using dot notation
b1.title = "Python 101"

# 2. Using setattr(object, attribute_name_string, value)
setattr(b1, "author", "John Doe")

print(b1.title, "by", b1.author)
# Output: Python 101 by John Doe
```

4.2 Removing Attributes

You can delete attributes from an object using the del keyword or the delattr() function.

Python

```
class Box:
    def __init__(self):
        self.width = 10
        self.height = 20

my_box = Box()
del my_box.width          # Deletes 'width' using del
delattr(my_box, "height") # Deletes 'height' using delattr

# Check if 'width' exists after deletion
print(hasattr(my_box, 'width'))
# Output: False
```

5. Inheritance Structure

Inheritance is a powerful feature of OOP that allows a new class to absorb (inherit) the attributes and methods of an existing class. It promotes code reusability.

5.1 Superclasses and Subclasses

- **Superclass (Parent/Base Class):** The class whose properties are being inherited.
- **Subclass (Child/Derived Class):** The class that inherits the properties. It can have its own additional properties and methods.

5.2 Inheritance Basics

To inherit from a class, place the parent class's name inside parentheses when defining the child class.

Python

```
# Superclass
class Animal:
    def eat(self):
        print("I can eat")

# Subclass inheriting from Animal
class Dog(Animal):
    def bark(self):
        print("Woof!")

my_dog = Dog()
my_dog.eat() # Inherited method from Animal
# Output: I can eat
```

```
my_dog.bark() # Method specific to Dog
# Output: Woof!
```

6. Types of Inheritance

6.1 Single Inheritance

A subclass inherits from only *one* superclass. (The Animal -> Dog example above is Single Inheritance).

6.2 Multiple Inheritance

A subclass inherits from *more than one* superclass at the same time. Python fully supports this.

- **MRO (Method Resolution Order):** When multiple parent classes have a method with the same name, Python uses MRO (Left-to-Right search) to decide which one to call.

```
Python
class Father:
    def skill(self):
        print("Gardening")

class Mother:
    def skill(self):
        print("Painting")

# Inherits from both Father and Mother
class Child(Father, Mother):
    pass

c = Child()
c.skill()
# Output: Gardening
```

7. Advanced OOP Concepts

7.1 Inheritance and Overriding

Method Overriding occurs when a subclass provides its own specific implementation of a method that is already provided by its superclass. The subclass's method effectively "overwrites" the parent's method for objects of the subclass.

Python

```
class Bird:
    def speak(self):
        print("Chirp chirp")

class Crow(Bird):
    # Overriding the parent's speak method
    def speak(self):
        print("Caw caw!")

my_crow = Crow()
my_crow.speak()
# Output: Caw caw!
```

7.2 Finding Class/Object Components

Python provides tools for **introspection** (examining objects at runtime to find out what they are made of).

- `type(obj)`: Returns the class that the object belongs to.
- `isinstance(obj, Class)`: Returns True if `obj` is an instance of `Class` (or its subclass).
- `issubclass(Class1, Class2)`: Returns True if `Class1` is a subclass of `Class2`.
- `obj.__dict__`: A built-in dictionary that stores all the instance variables of an object.
- `dir(obj)`: Returns a list of all attributes and methods available to the object.

8. Debugging OOP

Searching for Missing Class Components

When working with dynamic classes, attempting to access an attribute or method that does not exist will raise an `AttributeError`. Debugging this involves checking if the component actually exists before using it.

- **`hasattr(object, 'attribute_name')`**: Returns True if the attribute exists, False otherwise. It is the safest way to check for missing components to avoid crashing the program.
- **`getattr(object, 'attribute_name', default_value)`**: Attempts to get an attribute. If it's missing, it returns the `default_value` instead of throwing an error.

Python

```
class User:
    def __init__(self):
        self.username = "admin"

u = User()
```

```
# 1. Searching/Checking before using (Preventing AttributeError)
if hasattr(u, "email"):
    print(u.email)
else:
    print("Email component is missing!")
# Output: Email component is missing!

# 2. Using getattr with a safe fallback
user_age = getattr(u, "age", "Age not provided")
print(user_age)
# Output: Age not provided
```